
Multi-turn Reinforcement Learning with Language Models

Younghan Park

younghanpark@utexas.edu

College of Natural Sciences

The University of Texas at Austin

Krystal An

annyue@utexas.edu

Electrical and Computer Sciences

The University of Texas at Austin

Abstract

A broad use case of large language models (LLMs) is in goal-directed decision-making tasks, or “agent” tasks, where an LLM needs to not just generate probable completions for a given prompt, but rather make intelligent decisions over an extended period of multi-turn interaction to accomplish a task. However, current RL methods for LLMs largely focus on single-turn reward maximization (e.g., proximal policy optimization). Policies trained via single-turn approaches often fail to perform effective credit assignment (e.g., they fail to identify good actions that may lead to long-term future performance despite appearing suboptimal at a given step) and do not endow policies with information-seeking behavior, which is important in agent problems. In this project, we evaluate several reinforcement learning algorithms effectively accommodating multiple turns, long horizons, and delayed rewards. Due to the computational constraints, we don’t consider methods that fine-tuning LLMs with reward function. We focus on the setting where an utterance is a single action, while utilizing multiple candidate utterances from a frozen pre-trained LLM for maximization in the Bellman backup. For further work, we publicly release our code and data for this project.¹ We also attach a YouTube link to a demo video that introduces our work.²

1 Introduction

Reinforcement Learning (RL) can be applied to Large Language Models (LLMs) in various innovative ways to address the complex tasks and enhance their performance (Wang et al., 2024). A common application of Large Language Models (LLMs) involves tasks related to goal-directed decision-making, also known as “agent” tasks. In these scenarios, an LLM isn’t merely tasked with generating the most likely completions for a given prompt but rather with making decisions over multiple interactions to achieve a goal. However, current Reinforcement Learning (RL) methods for LLMs primarily focus on maximizing rewards within a single turn. Policies trained using these single-turn approaches often struggle with effective credit assignment, failing to recognize advantageous actions that may lead to long-term success despite seeming suboptimal in the short term. Additionally, these policies lack information-seeking behavior, which is crucial in solving agent-related problems. Abdulhai et al. (2023) have addressed the limitation of the LLMs in the multi-turn interaction, it introduced 8 different tasks that are specifically designed to test the various RL capabilities in the goal-oriented multi turn interaction.

More specifically, constructing a fluent and intelligent dialogue agent can be used for intuitive interface and automation of human-interactive tasks such as chatbot. By using the RL system, this will allows agents to automatically optimize the dialogue performance in multi-turn interaction over human behavior. However, training the RL system online where agents can learn by interacting

¹<https://github.com/younghanstark/multi-turn-rl>

²<https://youtu.be/xPCUbzo2v6w>

with real humans is not cost friendly and time-consuming. Existing Reinforcement Learning (RL) approaches for dialogue typically involve interaction with a learned model of human behavior, as noted by [He et al. \(2018\)](#). This method allows for the inexpensive generation of experience. However, this form of training can lead the dialogue agent to exploit the model, potentially resulting in the generation of non-intelligible language. Furthermore, in many real-world applications, the access to interactive environments is restricted to a fixed offline dataset of logged experiences. Under such constraints, standard RL algorithms often perform poorly, with issues such as divergence. A key challenge in offline RL is the management of out-of-distribution actions. Without constraints, the maximization over actions in the target value can easily lead to overly optimistic Q-value predictions for these out-of-distribution actions, which compromises the effectiveness of the training process. [Verma et al. \(2022\)](#) develop a chatbot AI for the dialogue task with reinforcement learning and proposed three different algorithms to minimize the deviation from the behavior policy so as to avoid errors due to distributional shift.

In this work, we started with [Abdulhai et al. \(2023\)](#) first, selected the text-based maze task which is tailored to evaluate the capability of managing environments where the agent must make decisions based on textual input [Côté et al., 2018](#). It particularly emphasizes the RL capabilities of trajectory stitching and credit assignment. We setup the dataset and framework on TACC, and conducted evaluation of the maze task in two settings, fully observed and partially observed, both using a BC(behavioral cloning) approach. Conducted evaluation of Monte Carlo return method. However, the computation of fine-tuning LLMs with reward functions was really heavy. Also, the codebase heavily utilizes jax for multi GPU processing, but it was a bit hard considering our previous experience in this field. Therefore, we tried to find a similar but new topic, and we decided to work on [Verma et al. \(2022\)](#) which doesn't involve heavy LLM finetuning and complex multi GPU training. In terms of CHAI negotiation bot for dialogue task, we modify the outdated codebase so that we can experiment the settings proposed in the paper, and review a few of possible algorithms to alternate with the algorithm they proposed in the paper.

2 Related Work

Reinforcement learning and large language models Starting from the technique called reinforcement learning from human feedback(RLHF) [Ziegler et al., 2019](#) [Ouyang et al., 2022](#), there have been a lot of successful approaches and methodologies in using RL-based approaches to state-of-the-art systems. However, most of the prior works are single-step bandit settings, while viewing the problem in a non-sequential way. Our topic is different from the previous approaches since we consider multiple turns of dialogue and RL algorithms that can be applied to long-term decision making.

Reinforcement learning in text-based games Reinforcement learning has shown success in text-based environments [Osborne et al., 2021](#). There are several challenges including partial observability, large state space, sparse action space, or long-term credit assignment, in this literature. In terms of large state space, [Yuan et al. \(2018\)](#) succeeded to mitigate this issue by crediting a bonus when encountering an unseen state. In terms of sparse action space, [Zahavy et al. \(2018\)](#) succeeded to deal with the issue with the technique called action elimination. The technique introduces a network called Action Elimination Network (AEN) that eliminates sub-optimal actions. This is an active research area and there are several challenges that haven't been solved.

Solution for deviation from behavioral policy in Offline Reinforcement Learning Offline reinforcement learning (RL) addresses the problem of learning effective policies entirely from previously collected data, without online interaction. To overcome this limitation, many papers have proposed different solutions. Paper [Xu et al., 2021](#) introduces the Constraints Penalized Q-Learning (CPQ) algorithm to address the challenge of learning safe and effective RL policies from offline datasets. CPQ modifies Q-learning by penalizing out-of-distribution actions and adjusting the Bellman update to focus only on safe actions, ensuring adherence to safety constraints. Paper [Kostrikov et al., 2021](#) introduces Implicit Q-Learning (IQL), a new method for offline RL that

optimizes policy without evaluating actions outside of the dataset. IQL alternates between fitting an upper expectile value function and backing it up into a Q-function, using a modified loss function in a SARSA-style TD update. The approach doesn't require explicit constraints or regularization of out-of-distribution actions during training, and the policy is extracted using advantage-weighted behavioral cloning.

3 Preliminaries

Title: iPhone 5S 16 GB black silver - \$135
Description: Great condition. No scratches. I upgraded to iPhone 7. Please reply with phone number if you are interested. Thank you.

Buyer: Hi, I would like to buy the iPhone 5S
CHAI: Hi, great! The phone is in great condition. I'm asking \$135.00, it's in mint condition.
Buyer: Does it come with a charger?
CHAI: Yes it comes with a charger, as I upgraded. It's in mint condition.
Buyer: How long have you been using this phone?
CHAI: Just a month. It's in great condition and has a charger included.
Buyer: Did you buy this phone secondhand or from the Apple store?

CHAI: I bought it brand new from Apple store. I only used it a month, I'm happy I did not have to use this one.
Buyer: I can buy it for \$100
CHAI: I'll take that if you can pick it up in a few hours.
Buyer: Where will you be at 5pm today?
CHAI: I'm at the Apple store so about an hour.
Buyer: Okay, I can meet you at the Apple store.
CHAI: Awesome!
Buyer: *accept*

Figure 1: Negotiation example from dataset, with a conversation between CHAI(Blue) and buyer(green).

The multi-turn task we evaluated is the negotiation dialogue CraigslistBargain task He et al., 2018. It consists of 6682 advertisements scraped from Craigslist. Figure 1 presents an example from the dataset showing a log history of buying an iPhone. In this set up, seller represents as agent, and buyer represents as environment. There are 4 different types of responses: message, offer, accept offer and reject the offer. After an accept or reject response is made, it will end the episode. Reward is given based on the listed price and scale by a constant factor of 10. Chatbot will also be penalized -20 if buyer rejects the offer. Markov Decision Process (MDP) is defined as a tuple (S, A, T, r, γ) . The individual states are written as $s = \{s_u, s_{type}, s_{price}, s_{context}\}$ and actions as $a = \{a_u, a_{type}, a_{price}\}$, refer to Table 1 where u denotes utterances, $type$ denotes action types, $price$ denotes fraction price and $s_{context}$ denotes the listing context.

s_u	utterance	Hello!
$s_{context}$	context	Selling a new mountain bike.
s_{price}	price	0.5 (of list price)
s_{type}	action_type	message

Table 1: Example of state space.

In this dialogue task, our goal is to find an optimal policy to maximize the expected return:

$$E_{\pi, \tau} \left[\sum_{t=0}^{\tau} \gamma^t r(s_t, a_t) \right]$$

where $r : S \times A \rightarrow \mathbb{R}$ is the reward, $\gamma \in (0, 1]$ is the discount factor. In online RL, D is pre-collected dataset. The workflow can be divided into two main phases: Generation and Evaluation, with each phase encompassing specific steps:

In *Generation Phase*: **Model Initialization:** Begin with a pre-trained language model like GPT-2, which has general knowledge of language from the training corpus. **Task-Specific Fine-Tuning:** Further train the model on a dialogue corpus tailored to the specific task to adapt it to relevant

language patterns and objectives. **Response Generation:** From a given buyer’s response, GPT-2 use the fine-tuned model to generate 5 candidate utterances along with 5 candidate prices.

In *Evaluation Phase*: **Critic Training (Q-function):** Develop a Q-function to evaluate the effectiveness of the generated responses towards the task goal. **Response Scoring:** Apply the Q-function to score each response, determining their potential in achieving the desired dialogue outcome. **Response Selection:** Rather than choosing the action with the highest Q-value, which leads to low diversity, adopt a soft Q-learning approach.

4 Experiment

4.1 Experimental Framework

All the computational process is conducted in a single NVIDIA A100 GPU. For the experiment, we start from fine-tuning GPT2 (Radford et al., 2019) with CraigslistBargain dataset (He et al., 2018). We use GPT2-medium with batch-size 16. We then use the finetuned GPT2 weight to generate candidate sentences and embeddings. After that, we train the RL agents with the candidate sentences and embeddings. Now we explain 3 different RL algorithms we reproduced.

EMaQ Expected-Max Q-Learning (Ghasemipour et al., 2021) is an algorithm in between the standard Q-function evaluation and Q-learning backup operators. EMaQ algorithm can be implemented practically in offline RL settings. In this approach, we use samples from a candidate sentence distribution for the target Q value without an explicit actor. The target Q function can be written in following way:

$$Q_{target}^{prop}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \mathbb{E}_{\{a_i\}^N \sim \mu} \left[\max_i \bar{Q}(s', a_i) \right]$$

CQL Conservative Q-Learning (Kumar et al., 2020) is an effective offline RL algorithm. The goal of this algorithm is solving limitations via a conservative Q function. The conservative Q function has good theoretically property, which is that it produces a lower bound to its true value. In practice, CQL augments the standard Bellman error as an objective function with a regularizer which is very easy to implement on top of preexisting actor-critic implementations. The Q-learning objective is:

$$J^{CQL}(\theta) = (Q_\theta(s, a) - Q_{target}(s, a))^2 + \alpha f^{CQL}(\theta)$$

And the regularizer is defined as:

$$f^{CQL}(\theta) = \mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_a \exp(Q_\theta(s, a)) - \mathbb{E}_{a \sim \mathcal{D}} [Q_\theta(s, a)] \right]$$

BRAC Behavior Regularized Actor Critic (Wu et al., 2019) focusing on the regulation of Q-function to minimize the impact of deviation. More specifically, using a price network $\pi_\phi(a_{\text{price}}|s_{1:t})$ can generates a Gaussian distribution of prices based on the conversational state history. This network is optimized to maximize the expected Q-value of the proposed prices while minimizing the Kullback-Leibler divergence D_{KL} between the proposal distribution π_ϕ and a baseline distribution π_B , which models prices as a univariate conditional Gaussian dependent on previous offers.

The objective for training the proposal network is formally expressed as:

$$\max_{\pi_\phi} \mathbb{E}_{s, a \sim D} \mathbb{E}_{a_0 \sim \pi_\phi, \mu} [Q(s, a_0)] - D_{KL}(\pi_\phi(\cdot|s) \| \pi_B(\cdot|s))$$

This setup ensures that the pricing strategy aligns closely with historically observed pricing data, thereby avoiding reliance on potentially unreliable Q-values derived from poorly sampled or unrepresentative price points. The target Q-value, $Q_{BRAC}^{\text{target}}(s, a)$, is computed by considering the rewards

and the discounted future values estimated from the next states and actions proposed by π_ϕ and another policy μ , reflecting a comprehensive approach to incorporating learned behaviors into the price proposal mechanism:

$$Q_{BRAC}^{\text{target}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s_0, a_0 \sim \pi_\phi, \mu} [\bar{Q}(s_0, a_0)]$$

4.2 Experimental Results

We benchmark three RL algorithms by using the agent as a seller. For evaluation, we consider 4 different buyer model proposed in [He et al. \(2018\)](#). The name of the buyer model means the reward grant criteria. The goal of the fairness buyer is trained to achieve equal outcome for two agents. The length buyer tries to make the conversation as long as possible, i.e. maximize the number of turns. The utility buyer aims to maximize its profit by minimizing the final price between the two. The supervised setting is the most naive approach, where the agent is trained to mimic human's trajectory.

		seller		
		EMaQ	CQL	BRAC
buyer	fairness	0.96	0.92	0.98
	length	0.8	0.8	0.86
	supervised	0.94	0.96	0.96
	utility	0.88	0.92	0.96

Table 2: Negotiation acceptance rate

In Table 2, we provide a mean acceptance rate over the whole set of scenarios. We can observe that the BRAC algorithm results in the highest acceptance rate. One different property that BRAC has is that it ensures to align its strategy with historical price data. Therefore, it could avoid using inaccurate Q-values from poorly sampled scenarios. Also, we can find another observation. Training according to the length of trajectory doesn't help much to the negotiation. We can know that we should keep the length of trajectory, or the time elapsed for the negotiation short in certain range.

		seller		
		EMaQ	CQL	BRAC
buyer	fairness	0.95±0.27	0.69±0.21	0.96±0.15
	length	0.80±0.34	0.63±0.26	0.89±0.24
	supervised	0.85±0.21	0.77±0.16	0.93±0.18
	utility	0.82±0.26	0.70±0.22	0.95±0.18

Table 3: Mean and standard deviation of buyer price

		seller		
		EMaQ	CQL	BRAC
buyer	fairness	0.97±0.24	0.72±0.20	0.97±0.06
	length	0.95±0.25	0.74±0.20	0.96±0.15
	supervised	0.89±0.12	0.78±0.16	0.95±0.17
	utility	0.91±0.19	0.73±0.18	0.97±0.16

Table 4: Mean and standard deviation of seller price

In this negotiation dialogue chat, there are two prices we need to consider. First one is the list price where the initial asking price set by the seller, and second one is target price which is the buyer hoping to pay. Each cell in Table 3 is represent a buyer price that display a mean and standard

deviation over the whole set of scenarios. The buyer price is calculated by:

$$\text{Buyer Price} = \text{Average} \left(\frac{\text{Final price}}{\text{Target price}} \right)$$

In Table 4, we present a mean and standard deviation of seller prices over the whole set of scenarios which is calculated as:

$$\text{Seller Price} = \text{Average} \left(\frac{\text{Final price}}{\text{List price}} \right)$$

We can see that from Table 3, Behavior-regularized Q-learning performs best across the whole table, while Conservative Q-learning performs the worst. This could be due to the conservative nature of CQL, which prioritizes safety by minimizing the expected return deviation from a behavior policy. This approach leads to less aggressive negotiation tactics, resulting in final prices that are often lower than the buyer’s target price.

From Table 4, we observed that EMAQ and BRAC methods generally show high ratios than CQL, indicating that seller price are quite close to the list price. Also the BRAC has the lowest standard deviation among the methods, particularly in the ‘Fairness’ and ‘Utility’ metrics. Above all of these, we know that BRAC helps maintain a strong bargaining position.

5 Discussion and Future Work

In the future, it will be beneficial if we can implement the BRAC algorithm to dynamically adjust the penalty for out-of-distribution (OOD) actions based on the distance of the sampled actions from the behavior distribution, rather than relying on fixed penalties or simple distributions. We could also introduce controlled exploration mechanisms, such as entropy maximization within the scope of seen data, could help the model explore more within the confines of the dataset without stepping too far into OOD territory.

Batch-Constrained deep Q-learning [Fujimoto et al., 2018](#) is another one we can possible use in the future as a alternative algorithms. It approaches a notion of batch-constrained though a generative model. In a given state, BCQ can generates plausible candidates actions with high similarity to the batch and selects the highest valued action through a learned Q-network, then bias the value estimate to penalize rare or unseen actions. This can a well-suited for addressing out-of-distribution challenges in reinforcement learning.

Although we state that we can effectively solve the long-horizon negotiation dialouge task with freezing LLM weights and training Q-learning agents with the embedding of sentences, it could possibly generate utterances that are not natural and of which have low perplexity. However, if we conduct proximal policy optimization ([Schulman et al., 2017](#)), the computational constraint comes again. In this context, another possible way of improvement is utilizing idea proposed from [Santacroce et al. \(2023\)](#). This paper introduces Hydra-RLHF that reduces the memory usage of PPO. By utilizing this algorithm, we can make the memory usage to be smaller than conducting supervised finetuning. With some additional off-the-shelf optimization technologies applied, it is not impossible to do word-level training and utterance-level training at the same time, with significantly small computational constraints.

6 Conclusion

In this report, we have explored the challenges associated with applying reinforcement learning (RL) techniques to large language models (LLMs) in the context of goal-directed decision-making tasks, also known as "agent" tasks. Recognizing these limitations, we have reproduced and surveyed several RL algorithms tailored to address the complexities of multi-turn decision-making and long-horizon planning without relying on fine-tuning LLMs with reward functions, thereby circumventing computational constraints. By framing each utterance as a single action and leveraging multiple candidate

utterances from a pre-trained LLM for maximization in the Bellman backup, the algorithms aim to enhance credit assignment and foster information-seeking behavior crucial for effective performance in agent tasks. Future efforts may delve deeper into the integration of contextual information, explore alternative reward structures, and investigate techniques for scaling RL algorithms to larger LLMs. By continuing to innovate in this space, we can unlock the full potential of LLMs in navigating complex decision landscapes and driving intelligent behavior in real-world applications.

References

- Marwa Abdulhai, Isadora White, Charlie Snell, Charles Sun, Joey Hong, Yuexiang Zhai, Kelvin Xu, and Sergey Levine. LMRL Gym: Benchmarks for Multi-Turn Reinforcement Learning with Language Models. *arXiv*, 2023.
- Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Ruo Yu Tao, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. TextWorld: A Learning Environment for Text-based Games. *IJCAI Workshop*, 2018.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, 2018. URL <https://api.semanticscholar.org/CorpusID:54457299>.
- Seyed Kamyar Seyed Ghasemipour, Dale Schuurmans, and Shixiang Shane Gu. Emaq: Expected-max q-learning operator for simple yet effective offline and online rl. *ICML*, 2021.
- He He, Derek Chen, Anusha Balakrishnan, and Percy Liang. Decoupling strategy and generation in negotiation dialogues. *EMNLP*, 2018.
- Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline Reinforcement Learning with Implicit Q-Learning. *ICLR*, 2021.
- Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *NeurIPS*, 2020.
- Philip Osborne, Heido Nõmm, and Andre Freitas. A Survey of Text Games for Reinforcement Learning informed by Natural Language. *TACL*, 2021.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://api.semanticscholar.org/CorpusID:160025533>.
- Michael Santacrose, Yadong Lu, Han Yu, Yuanzhi Li, and Yelong Shen. Efficient rlhf: Reducing the memory usage of ppo. *arXiv*, 2023.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv*, 2017.
- Siddharth Verma, Justin Fu, Mengjiao Yang, and Sergey Levine. CHAI: A CHatbot AI for Task-Oriented Dialogue with Offline Reinforcement Learning. *NAACL*, 2022.
- Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback. *ICLR*, 2024.

-
- Yifan Wu, George Tucker, and Ofir Nachum. Behavior regularized offline reinforcement learning. *NeurIPS Workshop*, 2019.
- Haoran Xu, Xianyuan Zhan, and Xiangyu Zhu. Constraints Penalized Q-learning for Safe Offline Reinforcement Learning. *AAAI*, 2021.
- Xingdi Yuan, Marc-Alexandre Côté, Alessandro Sordoni, Romain Laroche, Remi Tachet des Combes, Matthew Hausknecht, and Adam Trischler. Counting to Explore and Generalize in Text-based Games. *arXiv*, 2018.
- Tom Zahavy, Matan Haroush, Nadav Merlis, Daniel J. Mankowitz, and Shie Mannor. Learn What Not to Learn: Action Elimination with Deep Reinforcement Learning. *NeurIPS*, 2018.
- Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-Tuning Language Models from Human Preferences. *arXiv*, 2019.